

BP10 系列 SDK

蓝牙应用开发说明文档

V2.0

版本记录

版本	作者	日期	修改日志
V2.2	KK	2024-10-12	1. 完善外挂 PA/LNA 控制 IO 配置说明
V2.1	KK	2024-8-14	1. 增加 LNA/PA 控制 IO 配置介绍;
V2.0	KK	2024-6-12	1. 修改文档命名; 2. 增加 profile 版本介绍; 3. 增加设备类型说明; 4. 调整结构, 更新介绍内容, 删除多余的信息;
V1.2	KK	2022-3-7	1. 增加 目录索引;
V1.1	KK	2022-3-4	1. 更新 蓝牙频偏值相关说明;
V1.0	KK	2022-2-28	1. 内容完善更新;
V0.5	KK	2019-12-5	1. URL 编码在线网站的增加, 以及注意事项; 2. 新增 AEC 使用的注意事项;
V0.4	KK	2019-10-29	1. 增加 BLE 属性配置, Attribute flags 的参数说明; 2. 蓝牙频偏值相关说明;
V0.3	KK	2019-9-18	增加蓝牙名称(非英文格式)说明
V0.2	KK	2019-9-16	1. 完善 BLE 工具说明(增加 UUID128 的修改方式, 属性的定义和移植方式) 2. 调整某些截图和内容说明
V0.1	KK	2019-9-6	初版发布

Note: 此文档适用 BP10 系列 SDK 版本: Karaoke_SDK, 标准 SDK, TWS_SDK, 8Mb_SDK
不同 SDK 中有个别差异, 如有疑问请联系 FAE;

目 录

1. BP10 蓝牙协议版本说明	1
2. BP10 外挂 PA/LNA 芯片 IO 配置介绍	1
3. 蓝牙参数配置	3
4. 蓝牙地址获取方式	3
5. 蓝牙相关数据保存位置	3
6. 蓝牙配对记录清除说明	4
7. 蓝牙发射功率配置	4
8. 蓝牙频偏值相关说明	5
9. 蓝牙名称说明	6
10. 经典蓝牙单次连接超时时间	7
11. BP10 连接手机超时时间	7
12. BLE 地址、名称修改和配套工具的使用	7
13. BLE 的服务 UUID 使用 128 位配置	9
14. BLE 的服务属性特征值修改说明	9
15. BLE Peripheral 广播内容说明	10
16. BLE 配置介绍	10
17. 模式切换，开关蓝牙功能使用方法	11
18. 蓝牙可见性和上电回连说明	11
19. 蓝牙设备类型 COD 介绍	12
20. 经典蓝牙如何搜索周围蓝牙设备	17
21. 蓝牙 profile 支持说明	18
22. AVRCP 功能说明	19
23. 获取歌曲歌词功能说明	19
24. 使用 HID 来控制手机音量说明	20
25. 通话相关参数配置	20
26. 通话时音效算法支持	21
27. HFP 电池电量同步功能	21
28. 开启 HFP 协议，但是不需要接听电话功能	21
29. 蓝牙自动重连功能	21
30. 蓝牙连接成功、断开连接提示音	22
31. TWS 参数配置说明	22
32. TWS 组网条件设置说明	22
33. TWS 对箱应用场景说明	23
34. TWS Soundbar 应用场景说明	23

1. BP10 蓝牙协议版本说明

1. 蓝牙核心协议版本：V5.3
2. 蓝牙 Profile 版本支持如下：

Profile	Role	Version
A2DP	source	V1.3.2
	sink	V1.3.2
AVCTP	controller	V1.4
	target	V1.4
AVDTP	source	V1.3
	sink	V1.3
AVRCP	controller	V1.6.2
	target	V1.6.2
GAVDP	initiator	V1.3
	acceptor	V1.3
HFP	AG	V1.8
	HF	V1.8
HID	N/A	V1.1.1
HSP	AG	not support
	HS	V1.2
OPP	client	V1.2.1
	server	V1.2.1
PBAP	PCE	V1.2.3
	PSE	not support
RFCOMM	N/A	V1.2
SPP	N/A	V1.2

3. 标准 SDK，TWS_SDK 新发布版本都已经关联 BT_PROFILE_BQB_ENABLE 宏定义，方便产品进行 BQB 认证；

注：如 SDK 版本和上述版本不匹配，请联系 FAE 进行更新；

2. BP10 外挂 PA/LNA 芯片 IO 配置介绍

在使用外挂 PA/LNA 芯片时，BP10 支持 2 组控制 IO，代码配置如下：

1. 第一组：GPIO_A0/A1

代码配置：

```
(* (volatile unsigned long *) 0x400300AC) = 21;
GPIO_PortAModeSet(GPIOA0, 4); //radio_rxen GPIO_A0
GPIO_PortAModeSet(GPIOA1, 8); //radio_txen GPIO_A1
```

2. 第二组：GPIO_A22/A23

代码配置：

```
(* (volatile unsigned long *) 0x400300AC) = 18;
GPIO_PortAModeSet(GPIOA22, 4); //radio_rxen
GPIO_PortAModeSet(GPIOA23, 4); //radio_txen
```

3. 注意事项:

(1) IO 的连接方式:

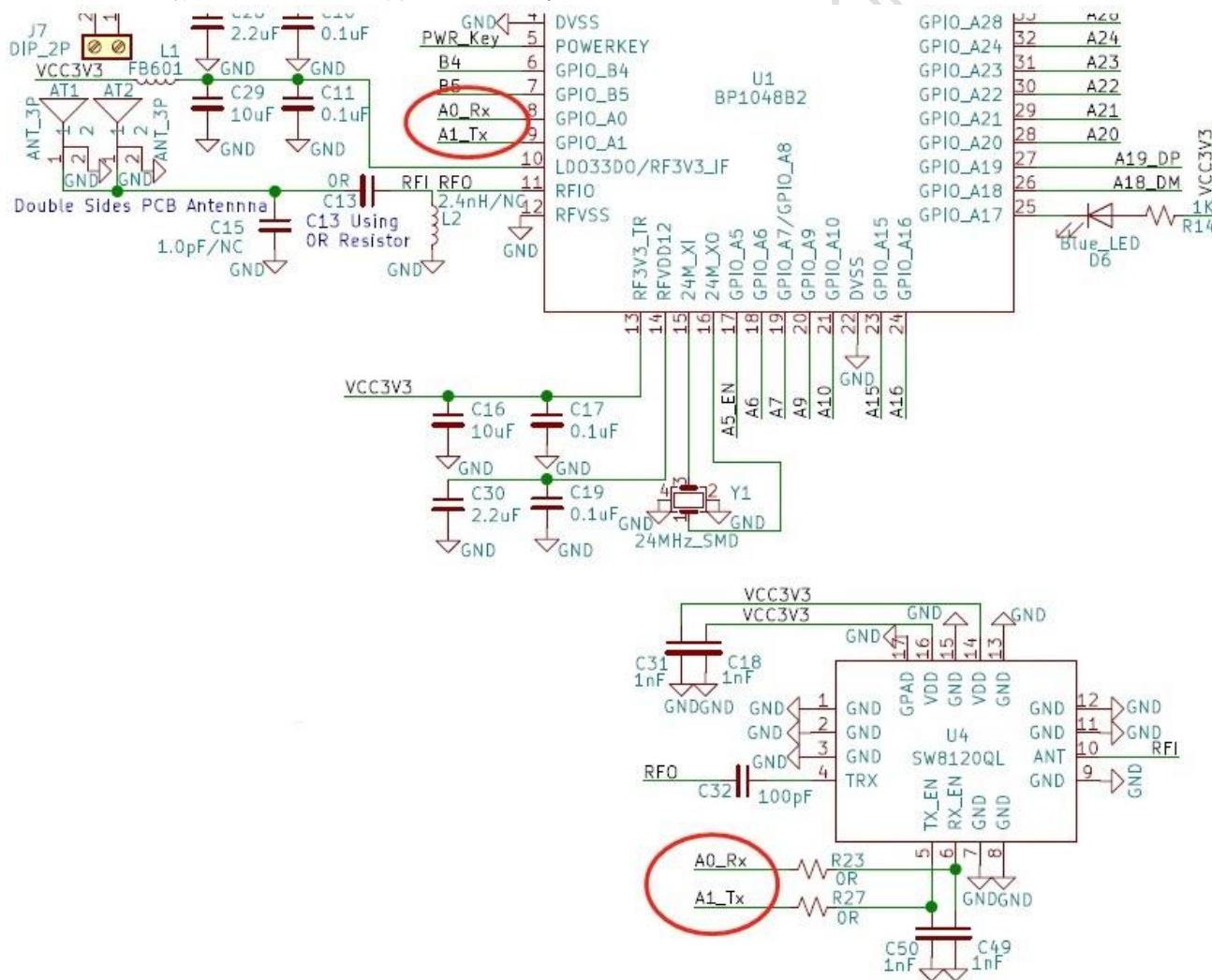
radio_rx 连接外挂芯片的 rx_en;

radio_tx 连接外挂芯片的 tx_en;

(2) 使用外部 LNA/PA 芯片时, 需要根据芯片的规格书, 调整 BP10 蓝牙的发射功率, 不能使用最大发射功率;

比如将 BP10 最大输出功率等级降低, 改为 15, 表格对应的功率为: -8dbm; 加上 PA 功率放大, 实际发射功率可能达到+18dbm; (实际的发射功率数据, 需要基于 LNA/PA 的芯片规格和硬件环境通过仪器来测定)

4. 以 A0/A1 做为 TX/RX 控制脚, 可以参考如下原理图:



3. 蓝牙参数配置

1. 蓝牙功能开关 `CFG_APP_BT_MODE_EN` 在 `app_config.h` 文件中;
2. 基带 EM 空间的分配参数配置在 `app_config.h` 文件中;
3. 蓝牙应用功能的配置, 蓝牙相关参数配置数据在 `bt_config.h` 文件中;

4. 蓝牙地址获取方式

1. 上电后从 `flash` 中指定位置读取蓝牙地址信息;
2. 如 `flash` 无数据, 从芯片的 `efuse` 中读取 `chip ID` 生成蓝牙地址, 然后保存到 `flash`;
3. 如 `efuse` 数据为空, 则通过随机数生成器生成蓝牙地址, 然后保存到 `flash`; (该方式基本无效, 只有在前期芯片没有烧录 `efuse` 信息的时候才有效, 后续芯片必然会烧录 `efuse`, 存在 `chip ID` 信息)

注: 用户可以不遵守如上的规则, 自行定义蓝牙地址;

4. 蓝牙地址的顺序说明:

- (1) 客户烧录的蓝牙地址的顺序是: **NAP-UAP-LAP**; 即保存到 `flash` 的顺序;
- (2) `flash` 内保存的蓝牙地址顺序: **NAP-UAP-LAP**, 同时手机和其他外设上看到顺序都是这个顺序;
- (3) 系统内部蓝牙在使用时, 地址的顺序为: **LAP-UAP-NAP**, 包含蓝牙连接手机成功后, 回调反馈的蓝牙顺序, 都是此顺序;

注: 此顺序和 `flash` 保存的顺序是反序的;

5. 蓝牙相关数据保存位置

5.1. 正常保存位置

1. 蓝牙地址, 蓝牙名称, BLE 地址, 频偏参数等蓝牙相关配置参数保存在 `flash` 的最后 4K, 即: `BT_CONFIG_ADDR` (`0x1FF000 ~ 0x1FFFFFF`); 该区域受 `download` 保护;
2. 蓝牙的配对记录区域: `BTDB_TOTAL_RECORD_ADDR` (`0x1FB000 ~ 0x1FCFFF`, 共 8K)
3. 蓝牙最后 1 次连接设备的地址: `BTDB_ALIVE_RECORD_ADDR` (`0x1FD000 ~ 0x1FDFFF`, 共 4K)
4. TWS SDK 中保存在 `flash` 中的蓝牙配对记录 9 个, 即 1 个 `tw`s 记录+8 个连接的手机记录; 标准 SDK 和 Karaoke SDK 中保存的蓝牙配对记录为 8 个, 即 8 个连接的手机记录;

5.2. 8Mb_SDK 保存位置

1. 蓝牙地址, BLE 地址, 频偏参数等重要蓝牙信息保存在 `flash` 的最后 4K, 即: `BT_CONFIG_ADDR` (`0x1FF000 ~ 0x1FFFFFF`); 该区域受 `download` 保护;

2. 蓝牙名称, BLE 名称, 蓝牙配置参数, 蓝牙配对记录保存位置偏移地址修改位置:
BT_Audio_APP\app_src\components\flash_manage\flash_table.h

```
#define BT_CONFIG_OFFSET      GET_MEMBER_OFFSET(reserve_1)
#define USER_CONFIG_OFFSET   GET_MEMBER_OFFSET(reserve_2)
#define FLASH_TABLE_OFFSET   GET_MEMBER_OFFSET(flash_table)    //数据索引表
#define BT_DATA_OFFSET       GET_MEMBER_OFFSET(bt_data)        //蓝牙配对记录数据区
#define BP_DATA_OFFSET       GET_MEMBER_OFFSET(bp_data)        //断点记忆数据区
#define EFFECT_DATA_OFFSET   GET_MEMBER_OFFSET(effect_data)    //音效参数区
#define REMIND_DATA_OFFSET   GET_MEMBER_OFFSET(remind)         //提示音参数区
#define SYS_PARA_OFFSET      GET_MEMBER_OFFSET(sys_parameter)  //系统参量化区域

void flash_table_init(void);
uint32_t get_remind_addr(void);    //提示音偏移地址
uint32_t get_effect_data_addr(void); //音效数据偏移地址
uint32_t get_bp_data_addr(void);   //断点记忆数据偏移地址
uint32_t get_bt_data_addr(void);   //蓝牙配对记录数据偏移地址
uint32_t get_user_config_addr(void); //用户数据区偏移地址
uint32_t get_bt_config_addr(void);  //蓝牙参数偏移地址
uint32_t get_sys_parameter_addr(void); //系统参量化数据偏移地址
bool flash_table_is_valid(void);
```

3. 蓝牙名称, BLE 名称, 蓝牙配置参数保存在系统参量化区域: SYS_PARA_OFFSET
偏移地址获取函数: `uint32_t get_sys_parameter_addr(void);`
4. 蓝牙配对记录 BT_DATA_OFFSET
偏移地址获取函数: `uint32_t get_bt_data_addr(void);`

6. 蓝牙配对记录清除说明

在 `bt_ddb_flash.h` 文件中, 调用 `BtDdb_Erase` 函数可以清除配对记录, 并不会再发起回连;

1. 将相关的系统变量清除, `btManager.btLinkDeviceInfo[]`
2. 将 `flash` 中 `BTDB_TOTAL_RECORD_ADDR` 位置的保存区域的数据清除;
3. 如需单独删除某个蓝牙记录步骤如下:
 - (1) 选择需要删除信息的蓝牙地址, 调用函数 `DdbFindRecord`, 找到对应的 `index` 位置;
 - (2) 调用函数 `DdbDeleteRecord(index)`, 将列表中的记录给清除;
 - (3) `btManager.ddbUpdate = 1;` 并将最新的列表数据, 保存到 `flash` 制定位置;

7. 蓝牙发射功率配置

在 `bt_config.h` 中, 配置 `BT_TX_POWER_LEVEL` 参数来调整发射功率;

1. 默认参数为 `level22(+6db)`;
2. 发射功率最大为 `level 23(+8db)`; 按照 `2db` 递减;
3. 发射功率过大, 可能会导致蓝牙回连会有干扰声的产生;

8. 蓝牙频偏值相关说明

1. 上电时，系统会从 flash 的制定区域读取频偏值，并使用；如果 flash 中没有保存频偏参数，则会使用默认频偏值 BT_TRIM(bt_config.h);
2. 可以通过串口日志信息观察当前板级的频偏值，蓝牙测试盒(v0.2.7 之后的固件版本)上也会显示校准后的参数；

串口日志表现如下：Freq trim:0x13

3. 在使用蓝牙测试盒的时候，可以根据实际情况选择操作：
 - (1) 频偏校准：将测试盒档位打到校准频偏模式(000)，在测试盒连接成功被测设备后，会自动将频偏值校准，并保存到 flash 中；测试盒上显示校准后的频率偏差值；
 - (2) 不需要频偏校准：将测试盒档位达到不校准频偏模式(001)，在测试盒连接成功被测设备后，不校准频偏，只是将当前被测设备的频率偏差值显示在屏幕上；
 - (3) 在频偏校准完成后，蓝牙测试盒上会显示校准后的频偏值；如下图，0x13 是校准值；4Khz 为校准后的偏差频率；



- (4) 若生产环节不需要每台机器做频偏校准，可使用试产的产品多个使用测试盒进行频率的校准后，确认测试盒显示的校准值是否保持接近，然后将频偏值写到 SDK 中；
- (5) 开始支持频偏值显示的 SDK 版本：TWS Beta19+P07，标准 SDK v0.1.12+P02
- (6) 如果蓝牙测试盒的频偏校正后面未显示参数，而是“---”，则说明频偏值超过一定范围，测试盒无法校准；

解决方案：

- ① 更换晶体；
- ② 增加或者调整 晶体负载电容；
- ③ 加大晶体谐振电流值；

例如：在 main 函数内，Clock_HOSCCurrentSet(10); //晶体谐振电流配置为 10

注：晶体的谐振电流参数配置可以咨询 FAE

4. 目前蓝牙测试盒是通过经典蓝牙的命令交互来进行频率的校准；部分客户只用到了 BP10 的 BLE，不需要经典蓝牙；此时需要通过蓝牙测试盒校准频偏；

解决方案：

- (1) 客户需要自定义个测试模式，将经典蓝牙的可见性状态配置为可被搜索可被连接状态，调用函数：BtSetAccessModeApi(BtAccessModeGeneralAccessible)；

- (2) 当串口日志上显示 APP_DBG("ACCESS MODE: WaitForPairing/Connection\r\n"), 则说明当前经典蓝牙状态设置成功;
- (3) 此时蓝牙测试盒可以正常的搜索和连接上 BP10, 进行频率校准;
- (4) 在测试完成后, 清除相应的标志, 将经典蓝牙可见性配置为 不可被搜索不可被连接状态, 调用函数: BtSetAccessModeApi(BtAccessModeNotAccessible), 并退出测试模式;

9. 蓝牙名称说明

9.1. 通用修改方式

1. 在 bt_config.h 头文件中配置蓝牙名称 BT_NAME
2. 蓝牙名称长度限定在了 40 字节;
如需要更长的字符需求, 则请联系 FAE;
3. 如蓝牙名称为中文, 或者其他语言, 请将内容进行 URL 编码, 再写入 bt_LocalDeviceName;

需要进行 URL 编码的数据可以在网站在线转换: <http://tool.chinaz.com/tools/urlencode.aspx>

注: 数字和字母不需要转换; 网上转换的数据为十六进制;

4. 中文名称, 可以直接在 bt_name.h 中, 定义中文名称, 但是需要在 bt_config.h 中, 屏蔽 BT_NAME 定义, 并打开 bt_name.h 的调用;

```

30
31 //蓝牙名称注意事项:
32 //1. 蓝牙名称支持中文, 需要使用URL编码
33 //2. BLE的名称修改在ble广播数据中体现 (ble_app_func.c)
34 //3. SDK蓝牙名称上电后从flash中读取, 如需使用固定的名称, 请移步到bt_app_func.c中LoadBtConfigurationParams函数内修改获取方式
35 //3. SDK蓝牙名称上电后从flash中读取, 如需使用固定的名称, 请移步到bt_app_func.c中LoadBtConfigurationParams函数内修改获取方式
36 #define BT_NAME "BP10_BT"
37 #include "bt_name.h"
38 #define BT_NAME_SIZE 40 //最大支持name size, 不能修改
39 #define BT_ADDR_SIZE 6

```

注: bt_name.h 中中文名称的修改, 尽量在 Andes 编译环境下进行修改; 避免由于文本格式错误, 导致修改异常;

9.2. 8Mb_SDK 修改方式

1. 此 SDK 将蓝牙名称存放到了参量化数据区, 进行统一管理; 目的: 方便后续进行名称的批量的修改;
2. 蓝牙名称修改路径: BT_Audio_APP\app_src\system_config\parameter.ini
3. 将 bt_LocalDeviceName 名称修改为指定的名称, 长度最长为 40Bytes; 如蓝牙名称为中文, 则直接修改为中文(ini 文件用 UTF-8 编码保存)即可;

注意: 在 bt_config.h 头文件中配置蓝牙名称 BT_NAME 暂时不生效了;

```

;-----
; parameter.ini 使用说明
; 半角分号后面的内容为注释
; 格式如下说明:
; parameter = 1 , 0
; parameter 前面为变量名
; 1      长度 (1个字节)
; 0      最后为变量赋值
;-----

; SDK蓝牙名称
bt_LocalDeviceName = 40, "BP10_BT"
; SDK BLE蓝牙名称 BLE的名称修改在ble广播数据中体现(ble_app_func.c)
ble_LocalDeviceName = 40, "BP10_BT_Ble"

```

4. 保存 parameter.ini 以后，点击运行 BT_Audio_APP\tools\flash.bat，会自动打包新的 img/mva 文件，提供给 下载/升级使用；
5. 客户如习惯之前的蓝牙名称修改方式，可以按照之前的逻辑自行修改即可；

10. 经典蓝牙单次连接超时时间

在 bt_config.h 中，BT_LSTO_DFT 可以配置蓝牙连接成功后，通讯异常，超时断开的的时间；
只有在 BP10 和手机连接时，角色为 Master 的情况下才有用；
注意：配置参数和实际参数的转换关系；

11. BP10 连接手机超时时间

BT_PAGE_TIMEOUT：可以设置每次 page 的超时时间；

12. BLE 地址、名称修改和配套工具的使用

1. BLE 的地址先从 flash 的指定位置读取，如无保存的地址信息，则通过 BT 的地址来生成，并保存到 flash；
2. 修改 BLE 的名称和 UUID 在 2 处地方：
 - (1) BLE 广播数据包的数组中 advertisement_data；

```

const uint8_t advertisement_data[] = {
    0x02, 0x01, 0x02, //flag:LE General Discoverable
    0x03, 0x03, 0x00, 0xab, //16bit service UUIDs
    9, 0x09, 'B', 'P', '1', '0', '-', 'B', 'L', 'E', // BLE名称
}; // 长度

```

- (2) 在 gatt 配置的相关参数内，需要使用 BLE 的 gatt_tool 工具，在 example.gatt 中修改配置；

```
example.gatt
1 PRIMARY_SERVICE, GAP_SERVICE
2 CHARACTERISTIC, GAP_DEVICE_NAME, READ, "BP10-BLE" BLE名称
3
4 PRIMARY_SERVICE AB00 UUID
5 CHARACTERISTIC, AB01, READ | WRITE | WRITE_WITHOUT_RESPONSE | DYNAMIC,
6 CHARACTERISTIC, AB02, NOTIFY | DYNAMIC,
7 CHARACTERISTIC, AB03, NOTIFY | DYNAMIC,
8
```

- (3) gatt_tool.exe 工具生成 out.h 中的 profile_data 数组，替换掉 ble_app_func.c 中的 profile_data；
- (4) 将 out.h 中 list service handle ranges 和 list mapping between characteristics and handles 的定义的参数同步到 ble_app_func.c，如下图所示：

```
//
// list service handle ranges
//
#define ATT_SERVICE_GAP_SERVICE_START_HANDLE 0x0001
#define ATT_SERVICE_GAP_SERVICE_END_HANDLE 0x0003
#define ATT_SERVICE_AB00_START_HANDLE 0x0004
#define ATT_SERVICE_AB00_END_HANDLE 0x000c

//
// list mapping between characteristics and handles
//
#define ATT_CHARACTERISTIC_GAP_DEVICE_NAME_01_VALUE_HANDLE 0x0003
#define ATT_CHARACTERISTIC_AB01_01_VALUE_HANDLE 0x0006
#define ATT_CHARACTERISTIC_AB02_01_VALUE_HANDLE 0x0008
#define ATT_CHARACTERISTIC_AB02_01_CLIENT_CONFIGURATION_HANDLE 0x0009
#define ATT_CHARACTERISTIC_AB03_01_VALUE_HANDLE 0x000b
#define ATT_CHARACTERISTIC_AB03_01_CLIENT_CONFIGURATION_HANDLE 0x000c
```

3. 工具的位置：MVsB1_Base_SDK\middleware\bluetooth\ble_adv_data_tool；使用方式请参阅该文件夹内的 readme.txt；
4. Gap_service 服务中，目前只是定义了 GAP_DEVICE_NAME 属性，假如需要增加属性，则自行添加；

```
assigned_uuids =
{
    'GAP_SERVICE' : 0x1800,
    'GATT_SERVICE' : 0x1801,
    'GAP_DEVICE_NAME' : 0x2a00,
    'GAP_APPEARANCE' : 0x2a01,
    'GAP_PERIPHERAL_PRIVACY_FLAG' : 0x2a02,
    'GAP_RECONNECTION_ADDRESS' : 0x2a03,
    'GAP_PERIPHERAL_PREFERRED_CONNECTION_PARAMETERS' : 0x2a04,
    'GATT_SERVICE_CHANGED' : 0x2a05,
}
```



```
PRIMARY_SERVICE, GAP_SERVICE
CHARACTERISTIC, GAP_DEVICE_NAME, READ, "BlX-BLE-Demo"
PRIMARY_SERVICE, B75C49D2-04A3-4071-A0B5-35853EB08307
CHARACTERISTIC, B85C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | DYNAMIC,
CHARACTERISTIC, BA5C49D2-04A3-4071-A0B5-35853EB08307, WRITE_WITHOUT_RESPONSE | DYNAMIC,
CHARACTERISTIC, B95C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | READ | WRITE_WITHOUT_RESPONSE | DYNAMIC,
```

在此插入GAP/GATT相关属性的内容

13.BLE 的服务 UUID 使用 128 位配置

1. 在 example.gatt 中修改配置如下图:

```
PRIMARY_SERVICE, B75C49D2-04A3-4071-A0B5-35853EB08307
CHARACTERISTIC, B85C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | DYNAMIC,
CHARACTERISTIC, BA5C49D2-04A3-4071-A0B5-35853EB08307, WRITE_WITHOUT_RESPONSE | DYNAMIC,
CHARACTERISTIC, B95C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | READ | WRITE_WITHOUT_RESPONSE | DYNAMIC,
```

2. 在 gatt_tool.exe 工具生成 out.h 中的 profile_data 数组, 替换掉 ble_app_func.c 中的 profile_data;
3. 将 out.h 中 list service handle ranges 和 list mapping between characteristics and handles 的定义的参数同步到 ble_app_func.c;
4. 在 ble_app_func.c 中, 同步修改 ble 广播数据包的数据内容:

```
const uint8_t advertisement_data[] = {
    0x02, 0x01, 0x02, //flag:LE General Discoverable
    0x11, 0x07, 0xB7, 0x5C, 0x49, 0xD2, 0x04, 0xA3, 0x40, 0x71, 0xA0, 0xB5, 0x35, 0x85, 0x3E, 0xB0, 0x83, 0x07, //128bit service UUIDs
    0x09, 0x09, 'B', 'P', '1', '0', '-', 'B', 'L', 'E', //
};
```

UUID128标识位 服务UUID128

14.BLE 的服务属性特征值修改说明

1. 在 example.gatt 中修改配置如下图:

```
PRIMARY_SERVICE, B75C49D2-04A3-4071-A0B5-35853EB08307
CHARACTERISTIC, B85C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | DYNAMIC,
CHARACTERISTIC, BA5C49D2-04A3-4071-A0B5-35853EB08307, WRITE_WITHOUT_RESPONSE | DYNAMIC, 根据实际的属性进行修改
CHARACTERISTIC, B95C49D2-04A3-4071-A0B5-35853EB08307, NOTIFY | READ | WRITE_WITHOUT_RESPONSE | DYNAMIC,
```

2. 属性如下: **BROADCAST**、**READ**、**WRITE_WITHOUT_RESPONSE**、**WRITE**、**NOTIFY**、**INDICATE**;
3. gatt_tool.exe 工具生成 out.h 中的 profile_data 数组, 替换掉 ble_app_func.c 中的 profile_data;
4. 将 out.h 中 list service handle ranges 和 list mapping between characteristics and handles 的定义的参数同步到 ble_app_func.c;

14.1. write/write without response

1. 手机通过此特征值发送 BLE Data 给到 BP10;
2. 在收到手机发送的数据后, 会通过回调函数 att_write 反馈到应用层, 可以通过注册的对应的 handle 获取手机发送的数据;

14.2. read

1. 手机通过此特征值来获取 BP10 发送的数据；
2. 在 read 的过程中，会通过回调函数 `att_read` 反馈到应用层，在对应的 handle 中，将需要发送给手机的数据，拷贝给 `buffer`，并返回实际的数据长度；

14.3. notify

1. BP10 通过 notify 直接发送 BLE 数据给到手机端；
2. 调用函数为：`att_server_notify`；
3. 使用注意事项：
 - (1) `att_server_notify` 函数需要在 `bt_stack_service` 的 runloop 中，增加 BLE 数据发送的循环处理；
 - (2) 不能在 `att_write` 内收到手机发送的数据时，直接调用 `att_server_notify`

15.BLE Peripheral 广播内容说明

1. 在 `ble_app_func.c` 中，`InitBlePlaycontrolProfile` 函数内，对 BLE 广播内容进行配置；
2. 当广播内容没有超过 32Bytes 时，则在 `advertisement_data` 中，对广播内容进行定义，并调用函数 `SetAdvertisingData` 进行广播；
3. 当广播内容超过 32Bytes 时，则需要将广播的数据进行拆分成 2 部分，每部分数据不能超过 32Bytes，赋值到数组 `gAdvertisementData` 和 `gResponseData` 中，并分别调用 `SetAdvertisingData` 和 `SetScanResponseData` 即可；

注：`gAdvertisementData` 和 `gResponseData` 的数据，必须要符合规范；

16.BLE 配置介绍

16.1. BLE 地址和经典蓝牙地址说明

根据不同的 BLE 使用场景，可以自行定义 BLE 地址和经典蓝牙地址是否保持一致，通过变量参数来进行配置 `g_playcontrol_app_context.use_random_addr_flag //0=public; 1=random address;`

16.2. BLE 广播间隔参数介绍

BLE 的广播间隔参数设置函数：`void SetAdvIntervalParams(uint16_t minValue, uint16_t maxValue);`
默认：`min=0x20, max=0x100;`

16.3. BLE 连接间隔参数介绍

当 BLE 外设和手机连接成功后，会通过回调事件 `BLE_CONN_UPDATE_COMPLETE`，将当前的连接间隔参数反馈到应用层，如：interval, latency, supervision_timeout;

16.4. BLE MTU 介绍

和手机连接成功后，如有配置 MTU 大小，则通过回调 `BLE_ATT_EXCHANGE_MTU_RESPONSE`，返回到应用层；
方便应用层发送数据的大小设置；

17. 模式切换，开关蓝牙功能使用方法

17.1. 不开关 RF 使用方法

1. 开启 `BT_FAST_POWER_ON_OFF_FUNC` 功能和蓝牙后台功能 `CFG_BT_BACKGROUND_RUN_EN`;
2. 功能开启后，在进入蓝牙模式，调用 `BtFastPowerOn` 开启蓝牙，进入可被搜索可被连接状态，并进行蓝牙回连；
3. 退出蓝牙模式时，调用 `BtFastPowerOff` 断开蓝牙连接，并使蓝牙进入不可被搜索不可被连接状态；

17.2. 开关 RF 使用方法

1. 关闭蓝牙后台功能 `CFG_BT_BACKGROUND_RUN_EN`;
2. 在退出蓝牙模式，调用 `BtPowerOff` 函数，断开蓝牙连接，复位基带，关闭蓝牙中断，删除蓝牙协议栈任务，关闭蓝牙模块；
3. 进入蓝牙模式，调用 `BtPowerOn` 函数，使能蓝牙模块时钟，初始化基带和蓝牙协议栈等；
4. 此方式下，系统电流会下降，对其他外设的干扰会降低；但是每次蓝牙启动时间会偏长；

18. 蓝牙可见性和上电回连说明

1. 在蓝牙初始化的时候，需要在蓝牙协议栈接收到 `MSG_BT_MID_STACK_INIT` 消息 (`bt_stack_service.c`)时，将 `GetBtManager()->btAccessModeEnable` 参数配置为 0，蓝牙会进入不可被搜索不可被连接状态；
2. 可以通过调用 `BTSetAccessMode` 函数，来修改蓝牙工作状态；

19. 蓝牙设备类型 COD 介绍

1. 蓝牙 COD（Class Of Device）是一个蓝牙设备分类类型值，由 24bit 组成，分别表示主要服务类别，主要设备类别和次要设备类别。

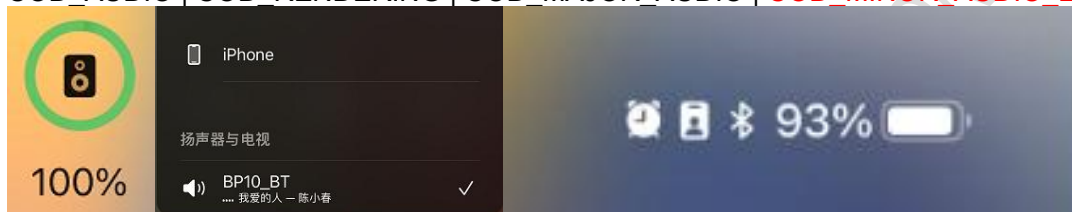


2. 设置方式：在函数 ConfigBtStackParams 函数内，使用 pCod 进行设备类型的配置；

19.1. COD 示例

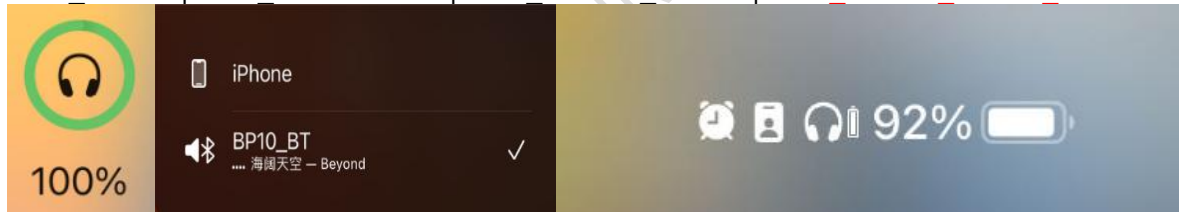
1. 音响设备：

COD_AUDIO | COD_RENDERING | COD_MAJOR_AUDIO | COD_MINOR_AUDIO_LOUDSPEAKER



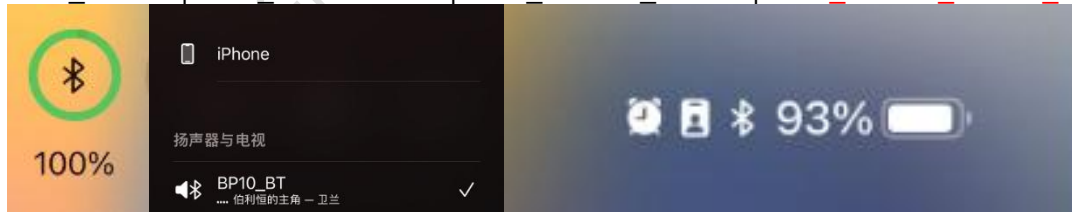
2. 头戴式耳机：

COD_AUDIO | COD_RENDERING | COD_MAJOR_AUDIO | COD_MINOR_AUDIO_HEADSET



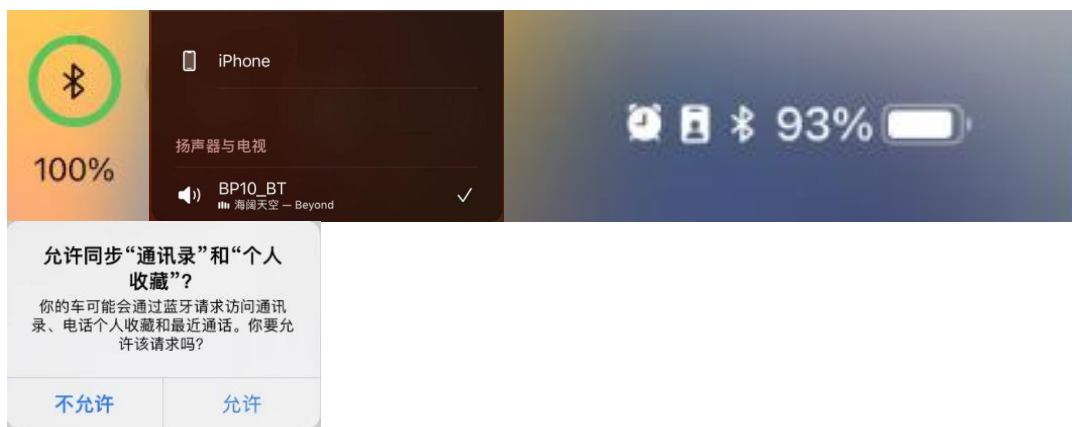
3. 高保真音频设备：

COD_AUDIO | COD_RENDERING | COD_MAJOR_AUDIO | COD_MINOR_AUDIO_HIFIAUDIO



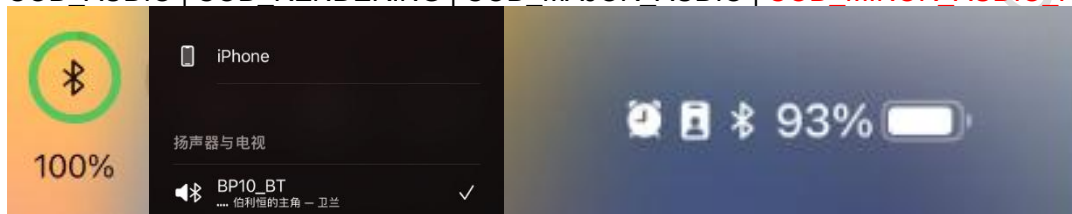
4. 免提设备：

COD_AUDIO | COD_RENDERING | COD_MAJOR_AUDIO | COD_MINOR_AUDIO_HANDSFREE



5. 便携式音频:

COD_AUDIO | COD_RENDERING | COD_MAJOR_AUDIO | COD_MINOR_AUDIO | PORTABLEAUDIO



6. 键盘外设:

COD_MAJOR_PERIPHERAL | COD_MINOR_PERIPH_KEYBOARD

19.2. COD 定义

23	13	12	8	7	2	1	0
Major Service Classes			Major Device Class		Minor Device Class		0b00
Bit	Class of Device Major Service Class						
13	Limited Discoverable Mode						
14	LE audio						
15	Reserved for future use						
16	Positioning (Location identification)						
17	Networking (LAN, Ad hoc, ...)						
18	Rendering (Printing, Speakers, ...)						
19	Capturing (Scanner, Microphone, ...)						
20	Object Transfer (v-Inbox, v-Folder, ...)						
21	Audio (Speaker, Microphone, Headset service, ...)						
22	Telephony (Cordless telephony, Modem, Headset service, ...)						
23	Information (WEB-server, WAP-server, ...)						

12	11	10	9	8	Major Device Class
0	0	0	0	0	Miscellaneous
0	0	0	0	1	Computer (desktop, notebook, PDA, organizer, ...)
0	0	0	1	0	Phone (cellular, cordless, pay phone, modem, ...)
0	0	0	1	1	LAN/Network Access point
0	0	1	0	0	Audio/Video (headset, speaker, stereo, video display, VCR, ...)
0	0	1	0	1	Peripheral (mouse, joystick, keyboard, ...)
0	0	1	1	0	Imaging (printer, scanner, camera, display, ...)
0	0	1	1	1	Wearable
0	1	0	0	0	Toy
0	1	0	0	1	Health
1	1	1	1	1	Uncategorized: device code not specified

2.8.2.4 Minor Device Class field – Audio/Video Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	0	Uncategorized, code not assigned
0	0	0	0	0	1	Wearable Headset Device
0	0	0	0	1	0	Hands-free Device
0	0	0	0	1	1	(Reserved)
0	0	0	1	0	0	Microphone
0	0	0	1	0	1	Loudspeaker
0	0	0	1	1	0	Headphones
0	0	0	1	1	1	Portable Audio
0	0	1	0	0	0	Car Audio
0	0	1	0	0	1	Set-top box
0	0	1	0	1	0	HiFi Audio Device
0	0	1	0	1	1	VCR
0	0	1	1	0	0	Video Camera
0	0	1	1	0	1	Camcorder
0	0	1	1	1	0	Video Monitor
0	0	1	1	1	1	Video Display and Loudspeaker
0	1	0	0	0	0	Video Conferencing
0	1	0	0	0	1	(Reserved)
0	1	0	0	1	0	Gaming/Toy

2.8.2.1 Minor Device Class field – Computer Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	0	Uncategorized, code for device not assigned
0	0	0	0	0	1	Desktop workstation
0	0	0	0	1	0	Server-class computer
0	0	0	0	1	1	Laptop
0	0	0	1	0	0	Handheld PC/PDA (clamshell)
0	0	0	1	0	1	Palm-size PC/PDA
0	0	0	1	1	0	Wearable computer (watch size)
0	0	0	1	1	1	Tablet

2.8.2.2 Minor Device Class field – Phone Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	0	Uncategorized, code for device not assigned
0	0	0	0	0	1	Cellular
0	0	0	0	1	0	Cordless
0	0	0	0	1	1	Smartphone
0	0	0	1	0	0	Wired modem or voice gateway
0	0	0	1	0	1	Common ISDN access

2.8.2.3 Minor Device Class field – LAN/Network Access point Major Class

Last Modified: 2022-05-25

7	6	5	Minor Device Class
0	0	0	Fully available
0	0	1	1% to 17% utilized
0	1	0	17% to 33% utilized
0	1	1	33% to 50% utilized
1	0	0	50% to 67% utilized
1	0	1	67% to 83% utilized
1	1	0	83% to 99% utilized
1	1	1	No service available

Last Modified: 2022-05-25

4	3	2	Minor Device Class
0	0	0	Uncategorized (use this value if no others apply)

2.8.2.5 Minor Device Class field – Peripheral Major Class

Last Modified: 2022-05-25

7	6	Minor Device Class
0	0	Uncategorized, code not assigned
0	1	Keyboard
1	0	Pointing device
1	1	Combo Keyboard/Pointing device

Last Modified: 2022-05-25

5	4	3	2	Minor Device Class
0	0	0	0	Uncategorized, code not assigned
0	0	0	1	Joystick
0	0	1	0	Gamepad
0	0	1	1	Remote control
0	1	0	0	Sensing device
0	1	0	1	Digitizer tablet
0	1	1	0	Card Reader (e.g. SIM Card Reader)
0	1	1	1	Digital Pen
1	0	0	0	Handheld scanner for barcodes, RFID, etc.
1	0	0	1	Handheld gestural input device (e.g., “wand” form factor)

2.8.2.6 Minor Device Class field – Imaging Major Class

Last Modified: 2022-05-25

7	6	5	4	Minor Device Class
X	X	X	1	Display
X	X	1	X	Camera
X	1	X	X	Scanner
1	X	X	X	Printer

Last Modified: 2022-05-25

3	2	Minor Device Class
0	0	Uncategorized, default

2.8.2.7 Minor Device Class field – Wearable Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	1	Wristwatch
0	0	0	0	1	0	Pager
0	0	0	0	1	1	Jacket
0	0	0	1	0	0	Helmet
0	0	0	1	0	1	Glasses

2.8.2.8 Minor Device Class field – Toy Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	1	Robot
0	0	0	0	1	0	Vehicle
0	0	0	0	1	1	Doll/Action figure
0	0	0	1	0	0	Controller
0	0	0	1	0	1	Game

2.8.2.9 Minor Device Class field – Health Major Class

Last Modified: 2022-05-25

7	6	5	4	3	2	Minor Device Class
0	0	0	0	0	0	Undefined
0	0	0	0	0	1	Blood Pressure Monitor
0	0	0	0	1	0	Thermometer
0	0	0	0	1	1	Weighing Scale
0	0	0	1	0	0	Glucose Meter
0	0	0	1	0	1	Pulse Oximeter
0	0	0	1	1	0	Heart/Pulse Rate Monitor
0	0	0	1	1	1	Health Data Display
0	0	1	0	0	0	Step Counter
0	0	1	0	0	1	Body Composition Analyzer
0	0	1	0	1	0	Peak Flow Monitor
0	0	1	0	1	1	Medication Monitor
0	0	1	1	0	0	Knee Prosthesis
0	0	1	1	0	1	Ankle Prosthesis
0	0	1	1	1	0	Generic Health Manager
0	0	1	1	1	1	Personal Mobility Device

20. 经典蓝牙如何搜索周围蓝牙设备

1. 开始搜索，请调用 `BTInquiry(INQUIRY_MODE_EXTENDED);`
2. 扫描到了设备信息后，会通过回调事件：
`BT_STACK_EVENT_COMMON_INQUIRY_RESULT` 反馈设备信息到应用层
3. 需要结束提前结束扫描，则调用 `BTInquiryCancel();`
4. 当次扫描结束，会通过回调事件反馈：
`BT_STACK_EVENT_COMMON_INQUIRY_COMPLETE`
5. 扫描取消后，会通过回调事件反馈：
`BT_STACK_EVENT_COMMON_INQUIRY_CANCELED`
6. 连接到搜到的蓝牙设备中的一个，并发起蓝牙的连接，可以参考如下代码：

```
void BtConnentRemoteDevice(uint8_t *addr)
{
```

```
btManager.btReconnectTryCount = 1;
btManager.btReconnectIntervalTime = BT_POR_INTERNAL_TIME;
btManager.btReconnectTimer.timerFlag = TIMER_USED;
btManager.btReconnectedFlag = 0;
memcpy(GetBtManager()->btDdbLastAddr, addr, 6);
btManager.btDdbLastProfile = (BT_PROFILE_SUPPORTED_HFP |
BT_PROFILE_SUPPORTED_A2DP | BT_PROFILE_SUPPORTED_AVRCP);
BtReconnectProfile();
}
```

7. 在蓝牙 Source 的 SDK 中，可通过经典蓝牙扫描结果，发起蓝牙连接，具体流程如下：

- (1) 调用蓝牙扫描：BtSourceInquiryStart
- (2) 扫描到的结果，会通过回调函数反馈到应用层，在 BtSourceInquiryResultParse 函数内进行体现，并缓存到本地；
- (3) 在扫描结束后，调用函数 BtSourceNameRequest，自动查询缓存的数据，蓝牙名称是否存在；如不存在，则会自动获取设备的蓝牙名称，调用 BTGetRemoteDeviceName；
- (4) BtSourceNameRequest 函数内，在全部检查完毕后，根据实际的需求，再连接对应的蓝牙设备；
- (5) 如在扫描过程中，需要停止，则调用 BtSourceInquiryStop 停止扫描过程；

21. 蓝牙 profile 支持说明

1. A2DP 和 AVRCP 必须要同时开启，由于考虑到 SDK 的使用情况，默认都是支持蓝牙音乐播放，所以必须要开启 A2DP；
2. BLE、HFP、SPP 根据实际的需要进行开启和关闭；
3. 支持不同的 profile，使用的 ram 资源是有区别的，请查看 bt_config.h 中 ram config 的相关说明；
4. 在 SDK 中，根据客户的蓝牙协议的需求，来自动调用对应的蓝牙库，以达到节省资源的目的，工程中会调用一个蓝牙库的复制脚本工具，在 BT_Audio_APP 目录下，有个 script_copy.ini 文件，客户可以根据实际的需求，来自行修改蓝牙库复制规则；

可参考下图的说明：

```
script_copy.ini
1 [main_config]
2 home_dir = ../../..
3 default_dest_lib = MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
4
5 [define_files]
6 files to search Macros
7 file1 = BT_Audio_APP/bt_audio_app_src/inc/app_config.h
8 file2 = BT_Audio_APP/bt_audio_app_src/inc/bt_config.h
9
10 [macro declare] 关联的宏定义
11 #include Macros names that should be used
12 BT_TWS_SUPPORT = 0
13 BLE_SUPPORT = 0
14 BT_HFP_SUPPORT = 0
15 BT_AVRCP_BROWSER_FUNC = 0
16 CFG_TWS_PEER_APP = 0
17 CFG_TWS_SOUNDBAR_APP = 0
18
19 [macro express]
20 ;expression maps libs should be copy 复制的规则
21 not BT_TWS_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Std.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
22 not BT_TWS_SUPPORT and not BLE_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Std_NoBle.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
23 not BT_TWS_SUPPORT and not BLE_SUPPORT and not BT_HFP_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Std_NoBle_NoHfp.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
24 not BT_TWS_SUPPORT and BT_AVRCP_BROWSER_FUNC = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Std_AvrpBrwa.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
25
26 BT_TWS_SUPPORT and CFG_TWS_PEER_APP and BLE_SUPPORT and BT_HFP_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Tws_Peer.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
27 BT_TWS_SUPPORT and CFG_TWS_PEER_APP and BLE_SUPPORT and not BT_HFP_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Tws_Peer.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
28 BT_TWS_SUPPORT and CFG_TWS_PEER_APP and not BLE_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Tws_Peer_NoBle.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
29 BT_TWS_SUPPORT and CFG_TWS_PEER_APP and not BLE_SUPPORT and not BT_HFP_SUPPORT = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Tws_Peer_NoBle_NoHfp.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
30
31 BT_TWS_SUPPORT and CFG_TWS_SOUNDBAR_APP = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Tws_SoundBar.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
32
33 [default_copy]
34 ;if no condition hits in macro express, do following copy
35 copy1 = MvSbl_Base_SDK/middleware/bluetooth/libBtStack_Std.a > MvSbl_Base_SDK/middleware/bluetooth/libBtStack.a
```

5. 如客户仅仅只需要 BLE 功能，不需要经典蓝牙，相关的 SDK 和修改方案请联系 FAE；

22.AVRCP 功能说明

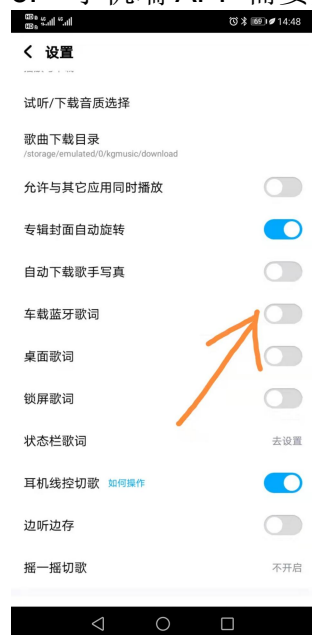
默认开启高级 AVRCP 功能：BT_AVRCP_ADVANCED

1. 通过高级 AVRCP 功能能及时更新当前的播放状态；
2. 开启 BT_AVRCP_VOLUME_SYNC，支持蓝牙音量同步功能，主要适用于苹果手机的播放音乐音量同步；
3. 开启 BT_AVRCP_SONG_PLAY_STATE，实时的刷新歌曲的播放时间；
4. 开启 BT_AVRCP_SONG_TRACK_INFOR，在歌曲开始播放、歌曲切换，获取歌曲的 ID3 信息；

23.获取歌曲歌词功能说明

手机部分音乐播放 APP(酷狗)支持歌曲歌词的获取，代码层面需要修改如下：

1. 在 bt_config.h 文件中，需要开启如下宏定义：
BT_AVRCP_PLAYER_SETTING ENABLE
BT_AVRCP_SONG_PLAY_STATE ENABLE
BT_AVRCP_SONG_TRACK_INFOR ENABLE
2. 调用函数 BTCtrlGetMediaInfor 来获取歌曲信息； 然后通过回调事件 BT_STACK_EVENT_AVRCP_ADV_MEDIA_INFO 反馈歌曲的信息；
3. 歌词信息会在 Title of the media 中显示；
4. 每次只能获取到 1 行歌词，此功能是由手机 APP 来决定；
5. 手机端 APP 需要开启 车载蓝牙歌词 功能，可以参考下图：



6. 在使用时，需要将歌词信息保存下来，不要直接打印，转到其他任务中再次处理，否则可能会导致蓝牙任务阻塞而出现异常；
7. 歌词获取的实时性策略已经进行了调整，如对歌词获取的速度有疑问，请联系 FAE

24.使用 HID 来控制手机音量说明

代码修改如下：

1. 开启 HID 协议的支持 `BT_HID_SUPPORT ENABLE`
2. 在手机连接成功后，主动发起 HID 连接，调用函数：`HidConnect(btManager.remoteAddr);`
3. 控制音量，调用函数 `HidCmd_VolUp / HidCmd_VolDn;`

注意事项：

- (1) `HidCmd_VolUp / HidCmd_VolDn` 函数内的延时时间需要为：`osTaskDelay(100);`
- (2) 音量控制的函数，不能在蓝牙协议栈任务下调用，其他任务调用皆可；

25.通话相关参数配置

1. 参数配置在 `bt_config.h` 中：

MIC 增益根据 MIC 的实际的灵敏度来进行调整：

`BT_HFP_MIC_PGA_GAIN`：默认值为 level 15(1.5db 左右)

`BT_HFP_MIC_DIGIT_GAIN`：默认值为 4095(0db)

`BT_HFP_INPUT_DIGIT_GAIN`：默认值为 4095(0db)，用于调小 DAC 输出的通话声音大小

2. 在 `audio_aec.h` 中，配置 AEC 的相关参数：

`BT_HFP_AEC_ECHO_LEVEL`：回音抑制，参数范围：0-5；默认设为 3；

`BT_HFP_AEC_NOISE_LEVEL`：噪声抑制，参数范围：0-5；默认设为 0，不开启；

配置 AEC DELAY 参数：

`MAX_DELAY_BLOCK`：默认值 32

`DEFAULT_DELAY_BLK`：默认值 8

注：

- (1) AEC 算法对 reference 数据和 MIC 数据的时差的要求不能超过 28ms；
- (2) 通话模式下，`samples_per_frame` 为 256 时(默认参数)，reference 和 mic 时差在 25ms 左右，`BT_HFP_AEC_DELAY_BLK` 可配置为 6；
- (3) `samples_per_frame` 为 128 时，时差在 17ms 左右，`BT_HFP_AEC_DELAY_BLK` 可配置为 4；
- (4) 如上时差、`delay_blk` 参数仅供参考，基于 128PIN 开发板测试；

AEC 功能注意事项：

- (1) 通话过程中，不能使用转采样功能；

- (2) MIC 增益相关参数不能太大，如饱和溢出，会导致 AEC 失效；
- (3) AEC ECHO 的参数配置越大，对回音的抑制效果越好，但是对双工的影响越大；
- (4) Reference 的数据一定要早于 mic 数据，给到 AEC 音效算法进行处理；否则会导致 AEC 算法失效；
- (5) 在 AEC 之后的 MIC 数据，尽量不要进行其他的音效处理算法处理，可能会由于算法的原因带入某些杂音；对方听到的声音不干净；

26. 通话时音效算法支持

目前在通话时，只支持 AEC 和变调算法；

1. 默认系统开启 AEC 算法；
2. 变调功能，通过 BT_HFP_MIC_PITCH_SHIFTER_FUNC 开启，调用 BtHf_PitchShifterEffectConfig 来改变声调；

27. HFP 电池电量同步功能

通过 BT_HFP_BATTERY_SYNC 开启电池电量同步功能

1. 在 HFP 协议连接成功后，自动同步当前设备的电量到手机端显示；
2. 需要配合 CFG_FUNC_POWER_MONITOR_EN 功能一起使用；

28. 开启 HFP 协议，但是不需要接听电话功能

通过 BT_HFP_MODE_DISABLE 开启该功能；

1. 在蓝牙连接成功后，正常的连接 HFP 协议；
2. 但是在呼入、呼出等通话功能开启的时候，设备端都会自动将通路切换到手机端进行操作；

29. 蓝牙自动重连功能

在 bt_config.h 头文件中开启 BT_RECONNECTION_FUNC 功能：

1. 开机回连(BT_POWERON_RECONNECTION)：回连次数(BT_POR_TRY_COUNTS)，间隔时间(BT_POR_INTERNAL_TIME)；
2. 连接丢失后自动重连(BT_BB_LOST_RECONNECTION)：回连次数(BT_BLR_TRY_COUNTS)，间隔时间(BT_BLR_INTERNAL_TIME)；

30. 蓝牙连接成功、断开连接提示音

在蓝牙连接成功和断开连接的时候，将 MSG_BT_STATE_CONNECTED 和 MSG_BT_STATE_DISCONNECT 2 个消息发送到 main_task.c 中，可在消息处理中加入相关的提示音；

31. TWS 参数配置说明

1. 在 app_config.h 中，需要打开宏定义 BT_TWS_SUPPORT，打开 TWS 功能；
2. TWS 应用场景分成 2 种，对箱模式和 Soundbar 模式；可以选择开启 CFG_TWS_SOUNDBAR_APP(Soundbar 模式) or CFG_TWS_PEER_APP(对箱模式)；
3. CFG_TWS_ONLY_IN_BT_MODE //enable:仅在蓝牙模式下工作; disable:在所有模式下工作
4. TWS_STATRT_PLAY_FRAM //设置 TWS music 通路的 delay 时间；此值越大，主副箱距离越远
5. CFG_EFFECT_MUSIC_MASTER // 0=master,slaver 独立有调音音效；1=只有 master 有调音，音效功能，slaver 仅做为接收；
6. TWS 回连参数说明：
 - BT_TWS_TRY_COUNTS : TWS 尝试重连次数 (开机)
 - BT_TWS_INTERNAL_TIME : TWS 重连每两次间隔时间(in seconds)
 - BT_TWS_BLR_TRY_COUNTS : TWS Link Loss 尝试重连次数
 - BT_TWS_BLR_INTERNAL_TIME : TWS Link Loss 重连每两次间隔时间(in seconds)
7. TWS_VOLUME_SYNC_ENABLE : 开启主从之间的音量同步控制
8. TWS_PAIRING_WHEN_PHONE_CONNECTED_SUPPORT //ENABLE: 表示手机连接时，该音箱可以发起组队； DISABLE: 表示手机连接时，该音箱无法发起组队
9. TWS_CONNECTE_WHEN_ACTIVE_DISCONNECTION_SUPPORT
 - //ENABLE: 表示用户主动断开 TWS 配对后，下次开机不能自动回连
 - //DISABLE: 表示用户主动断开 TWS 配对后，下次开机能再回连

32. TWS 组网条件设置说明

1. 组网条件：蓝牙名称匹配 or 自定义蓝牙条件，2 种方式可以自由组合进行组网
2. TWS_FILTER_NAME //组网蓝牙名称全匹配，第一次组网时，需要名称完全匹配才能组网成功；
3. TWS_FILTER_USER_DEFINED //组网自定义条件开关
 - TWS_FILTER_INFOR //自定义条件的设定，支持最长 6Bytes 的数据

33.TWS 对箱应用场景说明

对箱组网方式分为如下三种：

1. `TWS_PAIRING_MODE` `CFG_TWS_PEER_MASTER`
单键组网，发起组网的设备在组网成功后角色为 Master
2. `TWS_PAIRING_MODE` `CFG_TWS_PEER_SLAVE`
单键组网，发起组网的设备在组网成功后，角色为 Slave
3. `TWS_PAIRING_MODE` `CFG_TWS_ROLE_RANDOM`
双键组网，需要 2 个设备同时按键进入组网模式，才能组网成功；组网后的角色带有随机性；
4. TWS 组网，默认仅仅是在蓝牙模式下，未连上手机的情况下，允许组网；
5. `app_config.h` 中 `CFG_TWS_ONLY_IN_BT_MODE` `DISABLE` //在非蓝牙模式下允许组网；
6. `bt_config.h` 中 `TWS_PAIRING_WHEN_PHONE_CONNECTED_SUPPORT` `//ENABLE`
表示在手机连接成功后，允许 TWS 发起组网； `DISABLE` 表示在手机连接成功后，不允许 TWS 发起组网；

注：假如 TWS 组网需求不满足 或者 存在任何疑问，请联系 FAE。

34.TWS Soundbar 应用场景说明

`bt_config.h` 中参数配置说明如下：

1. Soundbar 应用，在开发初期，主副机的角色就已经确定，所以在开发时，主机和副机需要单独设定；
`TWS_PAIRING_MODE` `CFG_TWS_ROLE_MASTER` (主机配置)
`TWS_PAIRING_MODE` `CFG_TWS_ROLE_SLAVE` (副机配置)
2. 组网方式的选择：
`TWS_SIMPLE_PAIRING_SUPPORT` `//ENABLE`：单按键组网； `DISABLE`：双键组网